



Estruturas de Dados

Demonstração - RPN | Calculadora HP12C

Componentes:

Eric Gois | Yuri Rossi | Gabriel Lasinskais

Estrutura da Pilha



- Linguagem Python.
- RPN: Operadores vêm depois dos operandos.
- Fluxo de Memória: O topo da pilha é sempre o registrador X (o que vemos no visor). Abaixo dele estão Y, Z e T.
- Diferencial: Tratamento nativo para o padrão brasileiro de casas decimais.

Caso de Teste 1

Entrada RPN Inválida

```
Digite a expressão RPN: 10 +
```

```
-----  
Processando a expressão RPN: 10 +  
-----
```

```
> Token lido: '10'
```

```
Memória T: 0.0
```

```
Memória Z: 0.0
```

```
Memória Y: 0.0
```

```
Memória X: 10.0
```

```
> Token lido: '+'
```

```
Erro: Faltam números na pilha para usar o operador '+'.  
-----
```

- Entrada Simulada: 10 +
- O número 10 entra na pilha (Memória X).
- O operador + é lido.
- A lógica detecta que não há operandos suficientes na pilha (precisa de pelo menos 2 para somar).
- Saída: "Erro: Faltam números na pilha para usar o operador '+'."

```

Digite a expressão RPN: 10 2 + 3 * 6 - 5 /
-----
Processando a expressão RPN: 10 2 + 3 * 6 - 5 /
-----

> Token lido: '10'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 0.0
Memória X: 10.0

> Token lido: '2'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 10.0
Memória X: 2.0

> Token lido: '+'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 0.0
Memória X: 12.0

> Token lido: '3'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 12.0
Memória X: 3.0

> Token lido: '*'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 0.0
Memória X: 36.0

> Token lido: '6'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 36.0
Memória X: 6.0

> Token lido: '-'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 0.0
Memória X: 30.0

> Token lido: '5'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 30.0
Memória X: 5.0

> Token lido: '/'
Memória T: 0.0
Memória Z: 0.0
Memória Y: 0.0
Memória X: 6.0

-----
Expressão algébrica gerada: (((10 + 2) * 3) - 6) / 5)
Resultado Final: 6.0

```

Caso de Teste 2

$((10 + 2) \times 3 - 6) / 5$

- Entrada: 10 2 + 3 * 6 - 5 /

Memória	Passo 1 (Lê 10)	Passo 2 (Lê 2)	Passo 3 (Lê +)
T	0	0	0
Z	0	0	0
Y	0	10	0
X	10	2	12

- Resultado Final após processar toda a string: 6


```
Digite a expressão RPN: 1 1 0,03 4 * 30 / - / 1 - 100 *
```

```
-----  
Processando a expressão RPN: 1 1 0,03 4 * 30 / - / 1 - 100 *  
-----
```

```
> Token lido: '1'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 0.0  
Memória X: 1.0  
  
> Token lido: '*'  
Memória T: 0.0  
Memória Z: 1.0  
Memória Y: 1.0  
Memória X: 0.12  
  
> Token lido: '1'  
Memória T: 0.0  
Memória Z: 1.0  
Memória Y: 1.0  
Memória X: 0.12  
  
> Token lido: '0,03'  
Memória T: 1.0  
Memória Z: 1.0  
Memória Y: 0.12  
Memória X: 30.0  
  
> Token lido: '/'  
Memória T: 0.0  
Memória Z: 1.0  
Memória Y: 1.0  
Memória X: 0.004  
  
> Token lido: '-'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 1.0  
Memória X: 0.996  
  
> Token lido: '/'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 0.0  
Memória X: 1.0040160642570282  
  
> Token lido: '100'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 0.004016064257028162  
Memória X: 100.0  
  
> Token lido: '*'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 0.0  
Memória X: 0.40160642570281624  
  
-----  
Expressão algébrica gerada: (((1 / (1 - ((0.03 * 4) / 30))) - 1) * 100)  
Resultado Final: 0.40160642570281624  
  
> Token lido: '1'  
Memória T: 0.0  
Memória Z: 0.0  
Memória Y: 1.0040160642570282  
Memória X: 1.0
```

Caso de Teste 3

$$[(1 / (1 - (0,03 \times 4) / 30)) - 1] \times 100$$

- Entrada: 1 1 0,03 4 * 30 / - / 1 - 100 *
- O código processa a vírgula de 0,03 sem quebrar.
- A pilha atinge sua capacidade máxima de 4 níveis (T, Z, Y, X) antes da primeira multiplicação (*).
- Resultado Final: ~ 0.4016 (Exibido no visor / Memória X).

Conclusão

- A estrutura de Pilha é a solução natural para problemas RPN.
- O código garante a persistência da memória e visualização clara dos estados.
- Tratamento de erros robusto evita interrupções no fluxo do utilizador.

